

Session 7: Classifiers

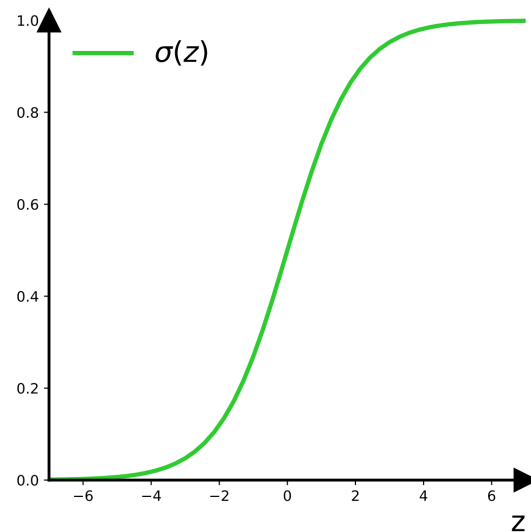
Supervised Learning: Classification

Q2: Derivative of the sigmoid

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$\sigma'(z) = ?$$

- A) $\sigma'(z) = \sigma(z) + \sigma(-z)$
- B) $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- C) $\sigma'(z) = \sigma(z)(1 + \sigma(-z))$
- D) $\sigma'(z) = \sigma(z)(1 - \sigma(-z))$



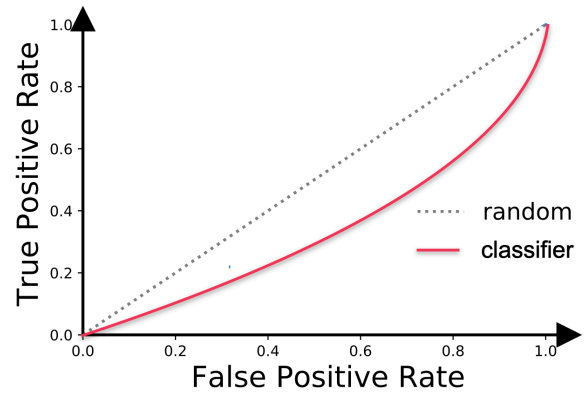
Q3: Binary cross-entropy loss derivation

For a perceptron $f(x) = \sigma(w^T x)$ to predict the prob of class 1, want to write $\mathcal{L}$ probabilistically to train with the negative log likelihood. Write down the loss as the **negative log likelihood** for a single training ex:

$$\mathcal{L}(x, y) = \begin{cases} -\log(1 - f(x)) & : y = 0 \\ -\log f(x) & : y = 1 \end{cases}$$

Q4: What would you do if you got a ROC curve like this? (Worse than random)

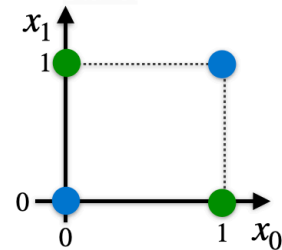
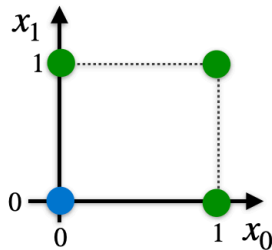
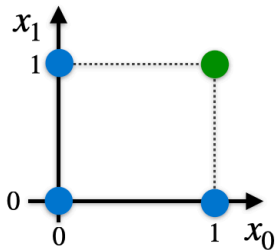
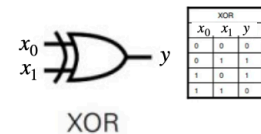
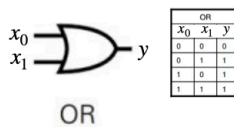
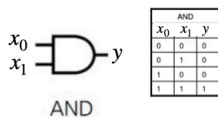
- A) Retrain the model
- B) Use the classifier with flipped labels
- C) Get more data



Q5: Perceptron limitations

Can you draw a boundary line between two examples ?

Inputs: $x = [x_0, x_1]$, $x_i \in [0, 1]$ **Outputs:** $y = [\text{off}, \text{on}]$



Theorem: A NN with _____ hidden layer(s) has the functional capacity to approximate any arbitrary Lipschitz continuous function.

Q6: Pytorch + conceptual

```
class myAwesomeModel(torch.nn.Module):
    def __init__(self, d, H):
        '''
        A 2 layer NN
        Inputs:
        - d: input dimension
        - H: # of hidden neurons
        '''
        super(myAwesomeModel, self).__init__()
        self.layer1 = torch.nn.Linear(d,H)
        self.layer2 = torch.nn.Linear(H,1)

    def forward(self, x):
        y = self.layer1(x)
        y = nn.Sigmoid()(y)
        y = self.layer2(y)
        return y

model = myAwesomeModel(d,H)
y = model(X) # X: shape (N, d)
```

Layers with learnable parameters {

Q: What type of model is this?

- A) Binary Classifier
- B) Multi-class classifier
- C) Regression